

DATA LABELING USING LINEAR CLASSIFIERS

1. Objectives

This laboratory is focused on designing a simple linear classifier able to automatically predict the label for unseen data applied as input to the system. The purpose of the laboratory is to help the student gain an intuition of the basic concepts of linear classifiers such as: nearest neighbor classification, k-nearest neighbor classification, data classification with multiclass support vector machine loss. By using the programming language Python, the students will implement from scratch a linear classifier that is able to predict the labels for a new set of data (never seen by the system).

2. Theoretical aspects

2.1. The Nearest Neighbor Classifier

One of the first techniques used to solve the traveling salesman problem was the nearest neighbor algorithm (*i.e.*, " Using a given list of cities and the distances between them determine the shortest possible route that allows visiting each city only once and returns to the origin city?"). In this approach, the salesman initiates the journey from a random city and systematically visits the closest city until all cities have been traversed. (Fig. 2.1).

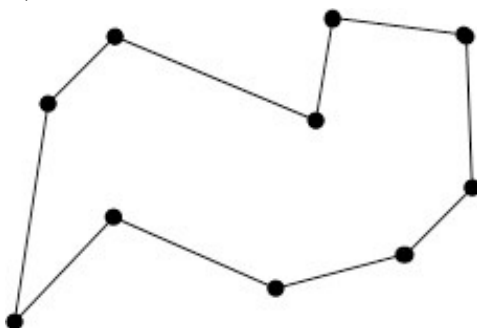


Fig. 2.1. The solution of a travelling salesman problem: the black line shows a possible loop that connects every black dot.

Below are the outlined the steps of the nearest neighbor algorithm:

1. initialize all vertices as unvisited;
2. select an arbitrary vertex, set it as the current vertex u . Mark u as visited;
3. find out the shortest edge connecting the current vertex u and an unvisited vertex v ;
4. set v as the current vertex u . Mark v as visited;
5. if all the vertices in the domain are visited, then terminate. Else, go to step 3.

In this laboratory we will apply the nearest neighbor classifier on one popular image database called **CIFAR-10**. This data set consists of 60,000 small images, with a resolution of 32 pixels in height and width. Every image is labeled with one of 10 classes such as “*airplane*”, “*automobile*”, “*bird*”, *etc.*

The collection of 60.000 images is partitioned into a training set, consisting of 50.000 images, and a test set, containing 10.000 images. If we consider the CIFAR-10 training set, which comprises 50.000 images (with 5.000 images for each label/class), and our goal is to classify the remaining 10.000 images to the corresponding category. The nearest neighbor classifier will take a test image, conduct a comparison with each of the training images, and predict the label based on the closest training image.

2.2. The k - Nearest Neighbor (k - NN)

It might seem unconventional to rely solely on the label of the nearest image for performing a prediction. In fact, better results are often achieved by employing a k -Nearest Neighbor Classifier.

The idea of the k -Nearest Neighbor classifier is straightforward: rather than identifying the single closest image in the training set, the technique will determine the **top k** closest images and let them collectively contribute to a maximum voting scheme to predict the label for the test image. Specifically, when $k = 1$, we obtain the Nearest Neighbor classifier. Intuitively, larger values of k parameter introduce a smoothing effect, rendering the classifier more resilient to outliers. (Fig. 2.2).

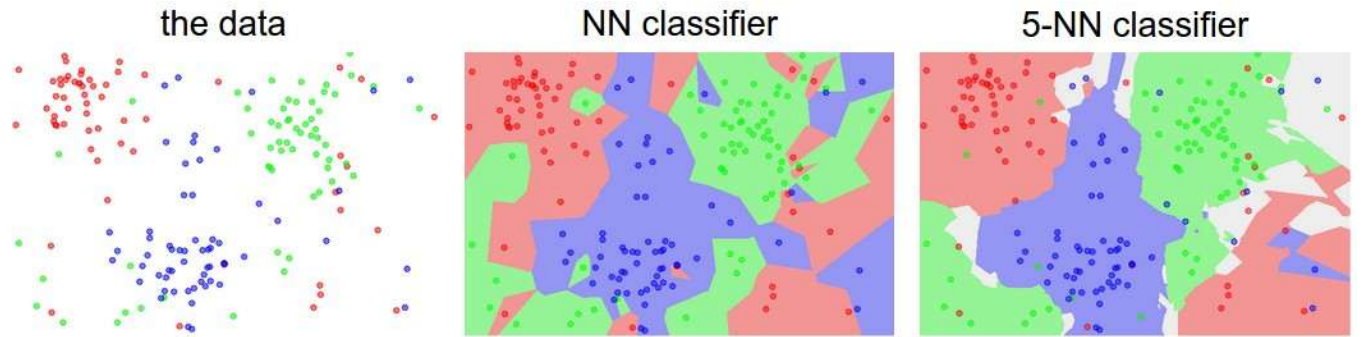


Fig. 2.2. An example of the difference between Nearest Neighbor and a 5-Nearest Neighbor classifier, using 2-dimensional points and 3 classes (red, blue, green) [1].

2.3. Linear Classifiers with SVM loss function

The SVM loss is designed to ensure that the classifier “wants” the correct class for each image to have a score higher than all the scores of the incorrect classes by a specified margin, Δ . The SVM can produce a specific outcome, aiming for a result that leads to a lower loss.

To be more specific on this point: Let us consider the i^{th} example existent in the training dataset that is characterized by its associated set of pixels x_i and the annotation label y_i that indicated the index of the correct class. The score function processes the pixels values and determines the vector $f(x_i, \mathbf{W})$ of class scores, which we will be denoted by $\mathbf{s}$ (the vector of scores). For example, the predicted score for the image x_i as belonging to class j is the j^{th} element in vector $\mathbf{s}$: $s_j = f(x_i, \mathbf{W})_j$. The Multiclass SVM loss for the i^{th} example is then formalized as follows:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta), \quad (1)$$

Our objective is to determine a set of weights allowing simultaneous prediction, for all the examples in the training dataset, the associated labels, while minimizing the overall loss value.

It can be observed that the SVM classifier uses linear scoring functions $f(x_i, W) = \mathbf{W} \cdot x_i$, so we reformulate the expression of the loss function as follows:

$$L_i = \sum_{j \neq y_i} \max(0, w_j^T \cdot x_i - w_{y_i}^T \cdot x_i + \Delta), \quad (2)$$

where w_j is the j^{th} row of $\mathbf{W}$ reshaped as a column. Nevertheless, the assumption may not be true for complex forms of the score function $f(\cdot)$.

To reduce the value of the loss function we can compute the value of the gradient with respect to the weights, indicating the direction in which the function experiences the steepest rate of increase. Nevertheless, the value of the gradient alone does not provide information with respect to the optimal distance to travel along this direction. Selecting the step size, also known as the *learning rate*, is one of the most crucial (and potentially challenging) tasks in the hyperparameters tuning during the classifier training. Fig. 2.3 illustrates the impact of the step size over the loss function.

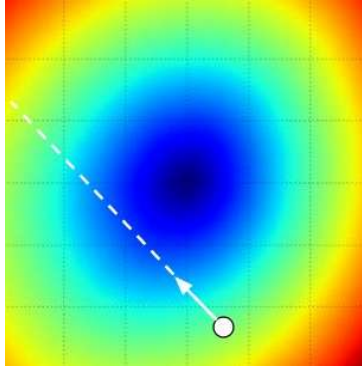


Fig. 2.3. Visualizing the effect of the step size [1]

We begin our analysis using a particular set of weights $\mathbf{W}$ (randomly initialized) and we compute the value of the gradient (or, more precisely, if it returns a negative value, represented by the white arrow in Fig. 2.3), that indicates the direction of the steepest decrease in the loss function. Incremental steps tend to yield steady but slower progress, while larger steps can result in more substantial advancements, but with a higher risk of error. It is important to observe that for a large step size, the risk of overpassing the minimum increases, and the new set of weights may enhance the loss value.

The gradient is calculated analytically using a direct formula (not numerical approximations). While this approach is computationally efficient, it can be more susceptible to errors during implementation.

Consequently, it is a common practice to compute the analytic gradient and compare it to the numerical gradient to verify the accuracy of the implementation. This is called a **gradient check**.

If we consider the SVM loss function applied for a single sample in the dataset (x_i):

$$L_i = \sum_{j \neq y_i} \max(0, w_j^T \cdot x_i - w_{y_i}^T \cdot x_i + \Delta) , \quad (3)$$

We can determine the derivative of the function with respect to the weights. If we determine the value of the gradient with respect to w_{y_i} in Equation 3 we obtain:

$$\nabla_{w_{y_i}} L_i = - \left(\sum_{j \neq y_i} 1(w_j^T \cdot x_i - w_{y_i}^T \cdot x_i + \Delta > 0) \right) \cdot x_i, \quad (4)$$

where $1(\cdot)$ is the indicator function that is one if the condition inside is true or zero otherwise. Although the expression may seem complex in written form, when translated into code, the derivative essentially involves counting the number of classes that did not meet the desired margin (thus contributing to the loss function). Consequently, data vector x_i scaled by this number determined the value of the gradient.

It is important to observe that the value of the gradient computed based on Equation 4 is determined with respect to the row of W that corresponds to the correct class. For the other rows W matrix where $j \neq y_i$ the gradient can be determined as follows:

$$\nabla_{w_j} L_i = 1(w_j^T \cdot x_i - w_{y_i}^T \cdot x_i + \Delta > 0) \cdot x_i , \quad (5)$$

3. Practical implementation

Pre-requirements: In order to start developing the applications and examples involved in the current laboratory platform in Python with PyCharm you need first to download and install **Python 3.13.7** from <https://www.python.org/downloads/release/python-3137/>.

For the Windows platforms, during the installation of the Python you have to check the box: “Add python.exe to PATH”.

The next step is to download the PyCharm IDE from <https://www.jetbrains.com/pycharm/download/> (free and open source). PyCharm is a dedicated Python Integrated Development Environment (IDE) providing a wide range of essential tools for Python developers, tightly integrated to create a convenient environment for productive Python, web, and data science development. During the PyCharm installation don't forget to check all the required boxes as presented in Fig. 3.1.

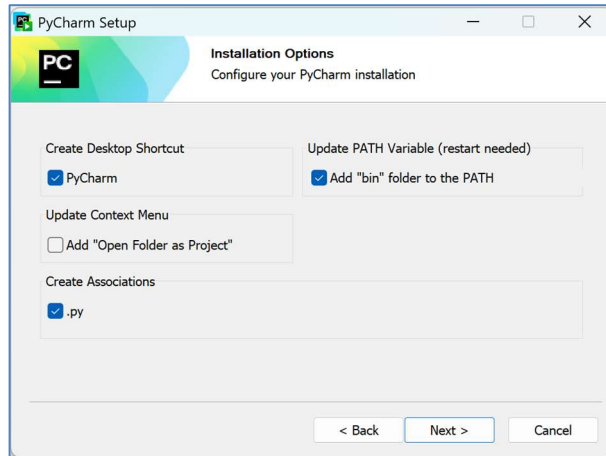


Fig. 3.1. PyCharm installation

Download the laboratory platform archive from the link provided by the instructor or from Moodle. Extract the archive, then copy the folder containing the .py files into your PyCharm working directory (PycharmProjects). Finally, open PyCharm and load the project using the **Open** option as in Fig. 3.2.

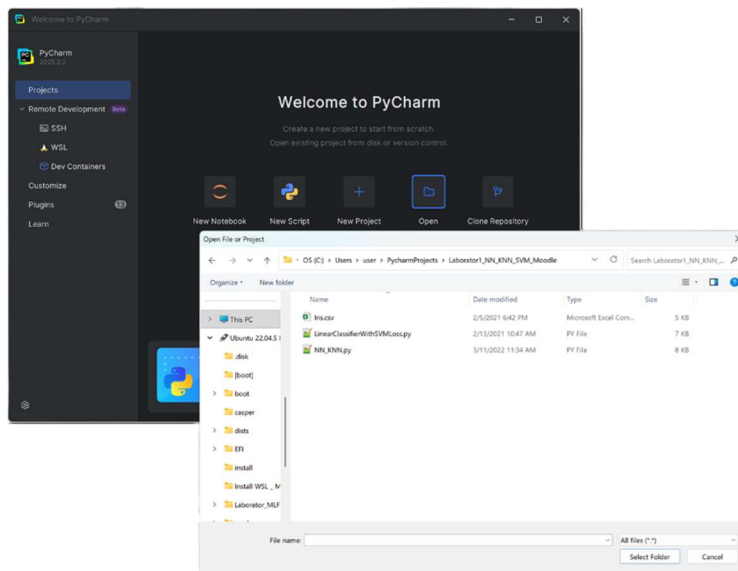


Fig. 3.2. Opening a project in PyCharm

Next, select the Python environment “AIenv” that is already created for the project. In PyCharm, go to Settings → Python → Interpreter, then on the right side choose Add Interpreter → Add Local Interpreter... In the window that opens, set Environment to *Select Existing* and locate the “AIenv” environment, as shown in Figure 3.3. The “AIenv” environment already includes the following libraries: opencv-python, tensorflow, pandas, and scikit-learn.

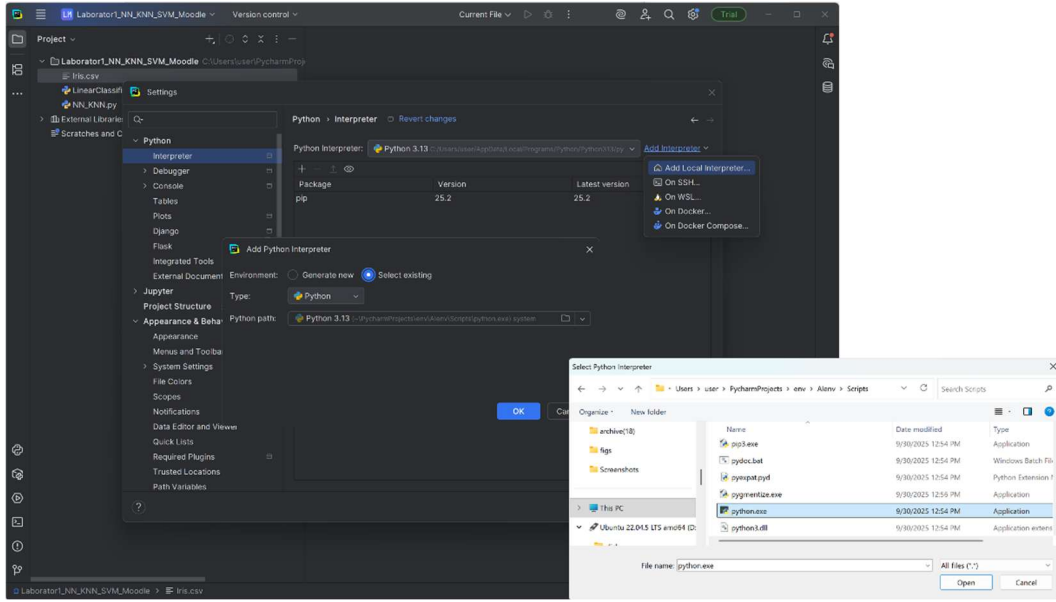


Fig. 3.3. Selecting the existing Python environment “AIenv” in PyCharm

Pre-requirements: Open the NN_KNN.py script. The current laboratory will use the CIFAR-10 dataset that consists of 60.000 color images with the resolution of 32 x 32 pixels, that are divided in 10 classes (0: airplane, 1: automobile, 2: bird, 3: cat, 4: deer, 5: dog, 6: frog, 7: horse, 8: ship, 9: truck), with 6.000 images per class. There are 50.000 images that can be used for training and 10.000 test images. The following example presents all the steps necessary to develop from scratch a nearest neighbor classifier able to predict the category for any image from the test dataset.

Application 1: Design a Nearest Neighbor classifier able to predict the category for any image from the test dataset. The system will compare the images, pixel by pixel and add up all the differences. In other words, given two images and representing them as vectors I_1 and I_2 , a naive choice for comparing them might be the L_1 distance:

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

where p represents the total number of pixels of an image. The two images are subtracted elementwise and then all differences are added up to a single number. If two images are identical the result will be zero. But if the images are very different the result will be large.

Step 1: Load the input data/labels (CIFAR-10) into Python as:

```
#Input data and Labels
```

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
```

Each row of the `x_train` / `x_test` data will contain one image of size 32 x 32 x 3.

Observation: If the following error occurs when loading the data set:

```
<results status="failed" reason="[SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed (_ssl.c:600)"
errno="13001"></results>
```

add into the script, the following instruction in the import section:

```
import ssl
ssl._create_default_https_context = ssl._create_unverified_context
```

Exercise 1: Determine the size of the four vectors (`x_train`, `y_train`, `x_test`, `y_test`) generated after loading the CIFAR-10 dataset.

Exercise 2: Visualize the first 10 images from the testing dataset with their associated labels.

Step 2: *Reshape the training and testing dataset* - The Nearest Neighbor classifier requires the data to be flattened out - one row per image in a 2D array and converted from *uint8* to *float64*. So, in order to compute the differences between two images the training and the testing dataset will be reshaped from a matrix structure to a vector of float elements.

```
np.float64(x_train.reshape(x_train.shape[0], 32 * 32 * 3))
```

Step 3: *Predict the labels for the first 200 images existent in the test dataset* – Each image from the 200 images test dataset will be applied as input to a function to obtain the predicted output.

```
for idx, img in enumerate(x_test_flatten[0:200]):
```

In order to perform the image prediction, define a function denoted:

`predictLabelNN(x_train_flatten, y_train, img)` that takes as input three parameters: the reshaped train dataset, the labels of the training dataset and the query image (test image). The function should return as output the predicted label for the current input (`return predictedLabel`).

Within the `predictLabelNN` function the following sub-steps should be performed:

- Read every image in the training dataset with its associated label;

```
for idx, imgT in enumerate(x_train_flatten):
```

- Compute the absolute difference between the image from the training dataset and the query image (test image);
- Add up all differences to a single number (`score`);

- d. Determine the nearest image from the training dataset and retain its associated label (`predictedLabel`).

Step 4: Compare the predicted label for the query image with the ground truth label – If the prediction is correct then increment the `numberOfCorrectPredictedImages` variable.

Step 5: Compute the system accuracy – In order to compute the accuracy, use the following relation:

$$Accuracy = 100 * \frac{Nr_of_correct_prediction}{No_of_images_considered_for_testing}$$

Exercise 3: Another common choice could be to use the L_2 distance instead of L_1 , which has the geometric interpretation of computing the Euclidean distance between two vectors. The distance takes the form:

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$

In other words, we would be computing the pixel-wise difference as before, but this time we square all of them, add them up and finally take the square root. Change the above code to determine the image similarity based on the L_2 distance. Determine the accuracy score in this case.

Observation: The scores are expected to be approximately 35%, depending on the images considered for testing. While the accuracy of this classifier is superior to random guessing (10%), it is still relatively modest. We could enhance the performance slightly by extending to a k-nearest neighbors' classifier. Instead of identifying the single closest image to our test set, this approach finds the k-closest images and determines a consensus in their labels, choosing the dominant class (the class with the highest number of occurrences).

Application 2: Design a K- Nearest Neighbor classifier able to predict the category for any image from the test dataset. The system will compare the images, pixel by pixel and add up all the differences. In other words, given two images and representing them as vectors I_1 and I_2 , a naive choice for comparing them might be the L_1 distance:

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

where p represents the total number of pixels of an image. The two images are subtracted elementwise and then all differences are added up to a single number. The system will determine the first k-images based on the similarity score and will determine the image label based on the maximum consensus between the classes returned.

Step 1: Create a new function denoted `predictLabelKNN(x_train_flatten, y_train, img)` – that takes as input three parameters: the reshaped train dataset, the labels of the training dataset and the query image (test image). The function should return as output the predicted label for the current input (`return predictedLabel`).

Within the `predictLabelKNN` function the following sub-steps should be performed:

- a. Read every image in the training dataset with its associated label;

```
for idx, imgT in enumerate(x_train_flatten):
```

- b. Compute the absolute difference between the image from the training dataset and the query image (test image);
- c. Add up all pixels' differences to a single number (`score`);
- d. Store all scores and the associated labels (for the test images) in a list denoted: `predictions = []` as pairs (`score, label`) - (`score, y_train[idx]`);
- e. Sort all elements in the `predictions` list in ascending order of priority based on the `score` parameter;

```
predictions = sorted(predictions, key=lambda x: x[0])
```

- f. Retain the top $k = 10$ prediction of the current image;
- g. Extract in a separate vector only the labels for the top k predictions;
- h. Determine the dominant class (the class with the highest number of occurrences) from the top k prediction list by calling the `most_frequent(list)` function (defined in the `NN_KNN.py` script).

```
def most_frequent(list):
    list = [x[0] for x in list]
    return [max(set(list), key = list.count)]
```

Step 2: In the main function replace the `predictLabelNN` function with the `predictLabelKNN` in order to call the new function.

Exercise 4: Modify the number of neighbors of the k-NN algorithm as specified in Table 1. What can be observed regarding the system accuracy? How about the prediction time? What will happen if we select $k = 1$?

Table 1. System performance evaluation for various numbers of neighbors

No of neighbors	3	5	10	20	50
System accuracy					

Exercise 5: Change the above code to determine the image similarity based on the L_2 distance. Determine the accuracy score in this case.

Table 2. System performance evaluation for various numbers of neighbors

No of neighbors	3	5	10	20	50
System accuracy					

Application 3: Design a linear classifier $f(x_i, \mathbf{W}) = \mathbf{W} \cdot x_i$ that uses the SVM loss function and is able to adjust, the random initialized set of weights $\mathbf{W}$, in order to predict the correct label (y_i) for a set of data applied as input (x_i).

Open the LinearClassifierWithSVMLoss.py script. The application will use as input a dataset with six points in the four-dimensional space (labeled as belonging to one of three possible classes) and a set of weights stored within the matrix ($\mathbf{W}$). The system will modify the elements of the $\mathbf{W}$ matrix so that the prediction ($\mathbf{W} \cdot x_i$) will be consistent with the set of ground truth labels (y_{train}). Finally, after the training (weights optimization), a new set of points (x_{test}) will be applied as input (test points) and the system will predict the class (label) for them.

Step 1: Consider the following training/testing datasets and the matrix of weights $\mathbf{W}$:

```
# Input points in the 4 dimensional space
x_train = np.array([[1, 5, 1, 4],
                    [2, 4, 0, 3],
                    [2, 1, 3, 3],
                    [2, 0, 4, 2],
                    [5, 1, 0, 2],
                    [4, 2, 1, 1]])

# Labels associated with the input points
y_train = [0, 0, 1, 1, 2, 2]

# Input points for prediction
x_test = np.array([[1, 5, 2, 4],
                   [2, 1, 2, 3],
                   [4, 1, 0, 1]])

# Labels associated with the testing points
y_test = [0, 1, 2]

# The matrix of wights
W = np.array([[-1, 2, 1, 3],
              [2, 0, -1, 4],
              [1, 3, 2, 1]])
```

Step 2: Take each input point and compute the scores s for all classes.

```
for idx, xsample in enumerate(x_train):
```

The scores are computed as $\mathbf{s} = \mathbf{W} \cdot \mathbf{x}_i$:

$$\mathbf{s} = \mathbf{W} \cdot \mathbf{x}_i = \begin{bmatrix} -1 & 2 & 1 & 3 \\ 2 & 0 & -1 & 4 \\ 1 & 3 & 2 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 5 \\ 1 \\ 4 \end{bmatrix} = \begin{bmatrix} 22 \\ 17 \\ 22 \end{bmatrix}$$

In the above example the point $\mathbf{x}_0$ is predicted with score $s_0 = 22$ as belonging to class 0, with the score $s_1 = 17$ to belong to class 1 and with score $s_2 = 22$ to belong to class 2.

In order compute the scores, define a function denoted: `predict(xsample, w)`, that takes as input two parameters: a train data point and the weights matrix and returns as output a vector of scores (`return s`).

Step 3: *Compute the loss for each data point (sample)* - The Multiclass SVM loss will be used. For the i^{th} data sample, the SVM loss is formalized as follows:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta)$$

In order compute the loss for a data point define a function denoted: `computeLossForASample(s, labelForSample, delta)` that takes as input three parameters: the score vector $\mathbf{s}$, the ground truth label of the current sample `y_train[idx]`, and delta (the SVM margin `delta = 1`) and returns as output the loss (`return loss_i`).

Step 4: *Compute the loss gradient* – The loss function gradient with respect to w_{y_i} can be computed as:

$$\nabla_{w_{y_i}} L_i = - \left(\sum_{j \neq y_i} 1(w_j^T \cdot x_i - w_{y_i}^T \cdot x_i + \Delta > 0) \right) \cdot x_i$$

where $1(\cdot)$ is the indicator function that is one if the condition inside is true or zero otherwise. Notice that this is the gradient only with respect to the row of $\mathbf{W}$ that corresponds to the correct class. For the other rows where $j \neq y_i$ the gradient is:

$$\nabla_{w_j} L_i = 1(w_j^T \cdot x_i - w_{y_i}^T \cdot x_i + \Delta > 0) \cdot x_i$$

To compute the loss gradient, define a function denoted:

`computeLossGradientForASample (W, s, currentDataPoint, labelForSample, delta)` that takes as input five parameters: the weights matrix $\mathbf{w}$, the scores vector $\mathbf{s}$, the current data point `x_train[idx]`, the label for the current data point `y_train[idx]`, and delta (the SVM margin `delta = 1`) and returns as output the loss gradient (`return dW_i`).

```
def computeLossGradientForASample(W, s, currentDataPoint, labelForSample, delta):

    dW_i = np.zeros(W.shape)
    syi = s[labelForSample]

    for j, sj in enumerate(s):
        distance = sj - syi + delta

        if j == labelForSample:
            continue

        if distance > 0:
            dW_i[j] = currentDataPoint
            dW_i[labelForSample] = dW_i[labelForSample] - currentDataPoint

    return dW_i
```

Step 5: *Compute the global loss (loss_L) for all the samples* – The global loss can be computed as the sum of losses loss_i determined for all samples.

Step 6: *Compute the global gradient loss matrix (dW)* – The global gradient loss matrix can be computed as the sum of gradient losses dW_i determined for all samples.

Step 7: *Compute the global normalized loss* – The normalized global loss can be computed as the ratio between the global loss (loss_L) and the number of input samples.

Step 8: *Compute the global normalized gradient loss matrix* – The global normalized gradient can be computed as the ratio between the global gradient loss (dW) and the number of the input samples.

Step 9: *Adjust the weights matrix (W)* – The weights matrix is adjusted using the following relation:

$$W = W - \text{step_size} * dW$$

where the `step_size` controls the gradient strength over the weights' adjustment process.

Exercise 6: Repeat the process, for the new set of weights, until the loss variation is inferior to 0.001. Determine the number of steps necessary for the algorithm to converge?

Exercise 7: Using the set of weights determined at Exercise 6, predict the labels for all the points existent in the `x_test` variable. Compare the results with the ground truth labels (`y_test`). What is the system accuracy?

Note: To retrieve the class with the maximum score `argmax()` function can be used!

Exercise 8: Starting from the code presented in Application 3: “LinearClassifierWithSVMLoss.py” modify the script to perform prediction on the Iris Flowers Dataset. The Iris Flowers Dataset involves predicting the flower species given measurements of iris flowers.

It is a multi-category classification problem. The number of observations for each class is balanced. There are 150 observations with 4 input variables and 1 output variable. The variable names are as follows:

1. Sepal length in cm.
2. Sepal width in cm.
3. Petal length in cm.
4. Petal width in cm.
5. Class (Iris Setosa, Iris Versicolour, Iris Virginica).

The data is stored in a *.csv file denoted: “iris.csv”. Read the csv file, *shuffle* the data and then split the data into `x_train`, `y_train`, `x_test`, `y_test`.

From the 150 observations existent in the *.csv file 120 elements will be used for training and 30 will be used for testing. The weights matrix W will be initialized with random numbers ranging from 0 to 1.

After performing the system training/evaluation specify:

- What is the optimal value for the weights adjustment step to obtain the maximum testing accuracy?
- What is the minimum number of steps necessary to train the system to obtain an accuracy score superior to 90%?
- Is the system influenced by the random initialization of the weights matrix? Justify your answer?
- Can this system reach 100% accuracy in the testing stage?

OBSERVATION: The laboratory report (*.pdf file) will contain:

- The code for all the exercises. The code lines introduced to solve an exercise will be marked with a different color and will be explained with comments.
- Print screens with the results displayed in the PyCharm console.
- The exercises solutions: tables of results and responses to the questions.

Bibliography:

[1]. <https://cs231n.github.io/classification/>